

Sampling Clock Impairments

A line-by-line mathematical explanation of the absolute-position polyphase resampler

The central idea

The algorithm generates output sample n at a nominal location P_n along the padded input. P_n is measured in polyphase units: if $U = \text{uprate}$, then U polyphase units equal one input-sample interval. Drift changes the spacing of the P_n sequence; jitter independently displaces the current sample from P_n to S_n .

Core clock model

Let $U = \text{uprate}$, $D = \text{drate}$, $\epsilon = \text{drift_ppm} \times 10^{-6}$, and $\sigma = U \times \text{jitter_ppm} \times 10^{-6}$.

$$P_0 = U / D$$

$$\Delta = D(1 + \epsilon)$$

$$P_n = P_0 + n\Delta$$

$$J_n \sim \text{Normal}(0, \sigma^2), \quad S_n = \text{clip}(P_n + J_n)$$

The polyphase filter bank then approximates the signal value at S_n/U input-sample intervals.

Units at a glance

Quantity	Meaning	Units
U	Number of polyphase branches	phases / input sample
D	Nominal movement per output	polyphase units / output
P_n, S_n	Locations along padded input	polyphase units
P_n / U	Same location	input-sample intervals
J_n	Independent timing displacement	polyphase units

1. Validate the mathematical domain

```
if not isinstance(uptate, (int, np.integer)) or uptate <= 0:
    raise ValueError(...)
```

U must be a positive integer because it is both the number of filter branches and the scale used to split a position into a whole-sample index and a phase index. Thus $U \in \{1, 2, 3, \dots\}$.

```
if not np.isfinite(drate) or drate <= 0.0:
    raise ValueError(...)
```

D must satisfy $0 < D < \infty$. It is a forward step through the input. A zero or negative D would stall or reverse the nominal clock.

```
if not np.isfinite(jitter_ppm) or jitter_ppm < 0.0:
    raise ValueError(...)
```

Jitter is specified as a standard deviation, so it must be finite and nonnegative. Zero means $J_n = 0$ exactly.

```
if not np.isfinite(drift_ppm):
    raise ValueError(...)
```

The signed drift value may be positive or negative, but it must be finite.

2. Convert drift from PPM into a clock step

```
nominal_position_increment = drate * (1.0 + drift_ppm * 1e-6)
```

One part per million is 10^{-6} . Therefore $\epsilon = \text{drift_ppm} \times 10^{-6}$, and the step from one nominal output position to the next is

$$\Delta = D(1 + \epsilon).$$

Positive drift makes Δ larger, so the algorithm moves through the input faster and usually produces fewer outputs. Negative drift makes Δ smaller and usually produces more outputs.

```
if not np.isfinite(nominal_position_increment) or nominal_position_increment <= 0.0:
    raise ValueError(...)
```

The loop requires $0 < \Delta < \infty$. In particular, $\epsilon \leq -1$ would make the step nonpositive.

3. Choose the random-number generator

```
rng = np.random.default_rng() if rng is None else rng
```

If the caller provides a generator, its state determines the sequence $J_0, J_1, \dots$. This makes jitter reproducible when the generator is seeded.

4. Construct the polyphase filter bank

```
taps_per_phase = int(np.ceil(len(h) / uprate))
h_pfb = partition_polyphase(h, uprate, taps_per_phase)
```

Let $N_h = \text{len}(h)$. The number of coefficients in each branch is

$$L = \text{ceil}(N_h / U).$$

The original FIR filter h is divided into U branches. Branch k , written h_k , represents one of U discrete fractional-delay choices. The phase index k ranges from 0 through $U - 1$.

Why polyphase filtering?

A desired position usually lies between input samples. Instead of explicitly upsampling by U and filtering a mostly-zero sequence, the bank selects only the branch corresponding to the needed fractional position. This computes the relevant interpolation efficiently.

5. Pad the input

```
padded_len = len(x) + 2 * taps_per_phase - 1
input_padded = np.zeros(padded_len, dtype=torch.complex64)
input_start = taps_per_phase - 1
input_padded[input_start : input_start + len(x)] = x
```

If $N_x = \text{len}(x)$, the padded length is $N_p = N_x + 2L - 1$. The original signal begins at padded index $L - 1$. This supplies zeros near both boundaries so every retained FIR window has L values.

In indexed form:

$$x_p[q] = x[q - (L - 1)] \text{ when } L - 1 \leq q < L - 1 + N_x, \text{ and } 0 \text{ otherwise.}$$

6. Determine the last representable location

```
max_input_idx = len(input_padded) - taps_per_phase
max_sample_position = max_input_idx * uprate + (uprate - 1)
```

A length- L FIR window beginning at integer index m uses $x_p[m]$, ..., $x_p[m + L - 1]$. Therefore the largest allowed start is

$$M = N_p - L.$$

At $m = M$, the latest phase is $U - 1$. Hence the latest absolute position is

$$P_{\max} = MU + (U - 1).$$

7. Initialize the clock and jitter scale

```
nominal_position = uprate / drate
jitter_std = uprate * jitter_ppm * 1e-6
```

The initial position is

$$P_0 = U / D \text{ polyphase units} = 1/D \text{ input-sample intervals.}$$

This is a preserved legacy alignment. When $U = D$, $P_0 = 1$ phase unit, or $1/U$ of an input-sample period. It is an alignment choice, not a drift term.

The jitter input is defined in millionths of one input-sample interval. Since one interval contains U phase units, its standard deviation becomes

$$\sigma = U \times \text{jitter_ppm} \times 10^{-6} \text{ polyphase units.}$$

Important separation

Drift changes Δ , the deterministic spacing of nominal positions. Jitter creates a new J_n for only the current output. Jitter is never added to P_{n+1} , so it does not form a random walk.

8. Estimate output storage

```
estimated_output_size = (
    int(np.ceil(len(input_padded) * uprate / nominal_position_increment)) + 1
)
```

The padded signal spans roughly $N_p U$ phase units. Dividing by step Δ estimates how many nominal positions fit:

$$N \approx \text{ceil}(N_p U / \Delta) + 1.$$

This is a storage estimate, not the exact mathematical output count. The exact loop condition uses $P_{\max}$ and P_0 .

```
output_samples = np.zeros(estimated_output_size, dtype=TorchSigComplexDataType)
max_output_samples = estimated_output_size * 16
output_idx = 0
```

The array initially has N slots. It may grow, but the $16\times$ ceiling prevents an accidental unbounded allocation.

Exact nominal count

When $P_0 \leq P_{\max}$, the nominal number of loop iterations is

$$N = \text{floor}((P_{\max} - P_0)/\Delta) + 1.$$

Jitter does not appear in this formula because it does not alter the loop's nominal trajectory.

9. Iterate over nominal sampling positions

```
while nominal_position <= max_sample_position:
```

At output n , the loop continues exactly when $P_n \leq P_{\max}$. Before the update at the bottom of the loop,

$$P_n = P_0 + n\Delta.$$

10. Draw the current timing error

```
timing_offset = (  
    rng.normal(0.0, jitter_std) if jitter_std > 0.0 else 0.0  
)
```

For $\sigma > 0$, draw an independent Gaussian value

$$J_n \sim \text{Normal}(0, \sigma^2).$$

For $\sigma = 0$, set $J_n = 0$ without consuming a random value.

11. Form and clamp the actual location

```
sample_position = np.clip(  
    nominal_position + timing_offset, 0.0, max_sample_position,  
)
```

First form the unconstrained jittered position $P_n + J_n$. Then project it onto the representable interval:

$$S_n = \min(P_{\max}, \max(0, P_n + J_n)).$$

Clamping guarantees a valid FIR start and phase. Near a boundary it changes the Gaussian distribution: all excursions beyond that boundary collapse onto the endpoint.

12. Split the absolute position into index and phase

```
input_idx, fractional_position = divmod(sample_position, uprate)  
input_idx = int(input_idx)  
phase = int(fractional_position)
```

Euclidean division writes the location as

$$S_n = m_n U + r_n, \quad 0 \leq r_n < U,$$

where $m_n = \text{floor}(S_n/U)$ is the whole input-sample index and r_n is the remainder in phase units. The code then chooses $k_n = \text{floor}(r_n)$ as the integer branch index.

Resolution

The continuous fractional remainder r_n is quantized to one of U branches. The discarded sub-phase fraction is less than one phase unit, or less than $1/U$ of an input-sample interval.

13. Select the FIR window and branch

```
delay_slice = input_padded[
    input_idx : input_idx + taps_per_phase
]
h_phase = h_pfb[phase][:taps_per_phase]
```

Using $m_n = \text{input_idx}$, $k_n = \text{phase}$, and $L = \text{taps_per_phase}$, the code selects

$$[x_p[m_n], x_p[m_n+1], \dots, x_p[m_n+L-1]]$$

and branch coefficients $[h_{k_n}[0], \dots, h_{k_n}[L-1]]$.

14. Compute the complex FIR output

```
pfb_out = np.sum(h_phase * delay_slice[::-1])
```

Reversing the input window gives the convolution-style dot product

$$y_n = \sum_{\ell=0}^{L-1} h_{k_n}[\ell] x_p[m_n + L - 1 - \ell].$$

The coefficients are real and x_p is complex. Therefore multiplication distributes over real and imaginary parts:

$$\begin{aligned} \text{Re}(y_n) &= \sum h_{k_n}[\ell] \text{Re}(x_p[m_n + L - 1 - \ell]), \\ \text{Im}(y_n) &= \sum h_{k_n}[\ell] \text{Im}(x_p[m_n + L - 1 - \ell]). \end{aligned}$$

Thus the one complex sum is mathematically equivalent to filtering I and Q separately.

15. Grow storage if the estimate was too small

```
if output_idx >= len(output_samples):
    if len(output_samples) >= max_output_samples:
        raise RuntimeError(...)
    new_size = min(2 * len(output_samples), max_output_samples)
    output_samples.resize(new_size, refcheck=False)
```

This block changes capacity only; it does not change y_n or any sampling position. Capacity doubles until it reaches the defensive maximum.

16. Store the output

```
output_samples[output_idx] = pfb_out
output_idx += 1
```

This assigns $\text{output_samples}[n] = y_n$, then advances n to $n + 1$.

17. Advance only the nominal clock

```
# Jitter is deliberately excluded: it affects only the current sample.  
nominal_position += nominal_position_increment
```

The recurrence is

$$P_{n+1} = P_n + \Delta = P_n + D(1 + \varepsilon).$$

Notice that J_n is absent. Consequently, a large jitter draw changes S_n but has no effect on P_{n+1} . This is independent timing jitter rather than accumulated phase noise.

18. Return only computed values

```
return output_samples[:output_idx]
```

If N outputs were computed, return $[y_0, \dots, y_{N-1}]$ and discard unused capacity.

Worked clock example

Take $U = 8$, $D = 8$, $\text{drift_ppm} = 100$, and ignore boundary clamping. Then

$$P_0 = U/D = 1, \quad \varepsilon = 0.0001, \quad \Delta = 8.0008.$$

In input-sample units, $P_0/U = 0.125$ and each output advances by $\Delta/U = 1.0001$ input samples.

n	P_n (phase units)	P_n/U (samples)	with example J_n	S_n	(m_n, k_n)
0	1.0000	0.1250	+0.30	1.3000	(0, 1)
1	9.0008	1.1251	-0.40	8.6008	(1, 0)
2	17.0016	2.1252	+0.90	17.9016	(2, 1)

The nominal sequence advances regularly by 8.0008. The jitter values move individual samples around that sequence. For example, $n = 1$ lands at 8.6008 phase units, so $\text{divmod}(8.6008, 8)$ gives whole index 1 and remainder 0.6008; converting the remainder to int selects phase 0.

The complete mathematical pipeline

1. Nominal clock

$$P_n = U/D + nD(1 + \text{drift_ppm} \times 10^{-6})$$

2. Independent jitter

$$J_n \sim \text{Normal}(0, [U \times \text{jitter_ppm} \times 10^{-6}]^2)$$

3. Representable sampling location

$$S_n = \text{clip}(P_n + J_n, 0, MU + U - 1)$$

4. Whole index and branch

$$m_n = \text{floor}(S_n/U), \quad k_n = \text{floor}(S_n - m_n U)$$

5. Polyphase interpolation

$$y_n = \sum_{\ell=0}^{L-1} h_{k_n}[\ell] x_p[m_n + L - 1 - \ell]$$

Interpretation

The algorithm walks through an absolute coordinate system attached to the padded input. Drift stretches or compresses the spacing between nominal sampling locations. Jitter briefly nudges each location. Euclidean division converts the resulting coordinate into an integer input window and a fractional-delay branch, and that branch performs the interpolation.

Two details to keep mentally separate

Position units versus input-sample units. P_n and S_n are in polyphase units. Divide by U to express them in input-sample intervals.

Initial alignment versus step size. $P_0 = U/D$ sets where sampling starts. $\Delta = D(1 + \epsilon)$ sets how far the clock moves after every output. Drift belongs only in Δ .

Compact pseudomathematical form

```
for n = 0, 1, ... while P_n <= P_max:
    J_n ~ Normal(0, sigma^2)
    S_n = clip(P_n + J_n, 0, P_max)
    (m_n, r_n) = divmod(S_n, U)
    k_n = floor(r_n)
    y_n = dot(h[k_n], reverse(x_p[m_n : m_n + L]))
    P_(n+1) = P_n + Delta
```

That is the entire function: generate a clock location, perturb it, decompose it, interpolate there, and advance the unperturbed clock.